

Everything You Need to Know to Do the Four Day 3 Tasks

A plain-language explanation of every retrieval, integration and observability idea the Day 3 hands-on tasks depend on — with a picture for each one. No prior experience of RAG systems required.

The one sentence the whole day rests on. When a retrieval system gives a wrong answer, **the overwhelming majority of the time the right passage was never retrieved.** The model then did exactly what it should — answered from what it was given. Everything in this document follows from that: ingestion quality caps retrieval quality, retrieval quality caps answer quality, and you cannot tell which stage failed unless you measure them separately.

Part	Supports	Concepts covered
Part 1	Task 1 — The ingestion bench	Tokens and what they cost, the context window, lost-in-the-middle, chunking strategies, overlap, four levels of extraction, the metadata that earns its place, versioning and supersession, token budgeting
Part 2	Task 2 — Retrieval benchmark	Dense, sparse and hybrid retrieval, reciprocal rank fusion, reranking and why two stages, metadata filtering, query rewriting, structural retrieval patterns, no-answer detection
Part 3	Task 3 — MCP servers	What MCP is and the integration arithmetic, client and server roles, tool contracts, when not to use MCP, the six security controls, and the confused deputy
Part 4	Task 4 — Improve and observe	The eight evaluation metrics, the diagnosis table, building a gold set, traces and spans, the observability dashboard, regression testing, and the thirteen improvement levers
Appendix	All four tasks	Formulas and checklists, and a glossary

How to read a section. Each concept has the same three parts: **what it is** in one or two sentences, **a picture**, and **why the task needs it**. If you are short of time, read the figures and their captions — they carry most of the content on their own.

1.1 What a token is, and why it is a commercial concept

A token is roughly three-quarters of a word in English, and considerably less in technical text full of part numbers and abbreviations. Models read, write and are billed in tokens.

Figure 1 · You are paying for retrieved context, not for the question

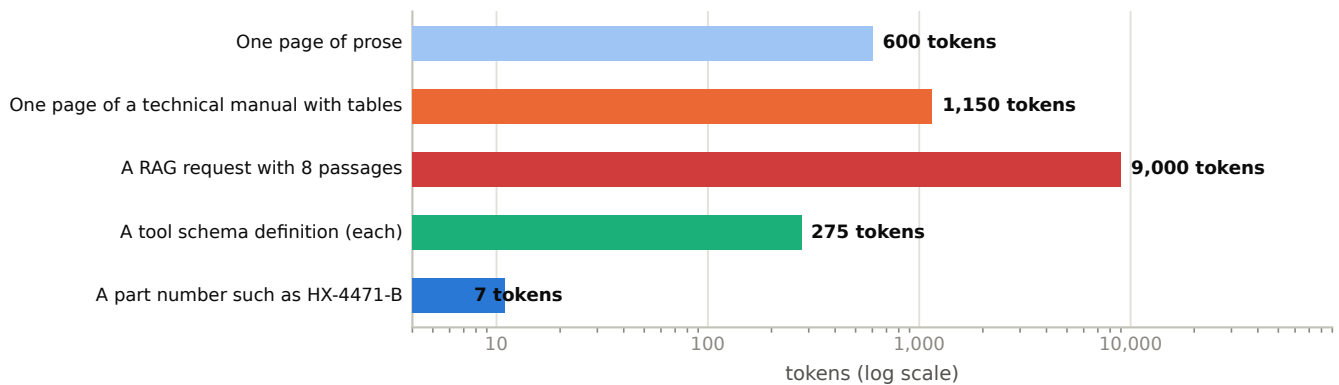


Figure 1. Note where the volume is. A user's question is negligible; the retrieved passages dominate the bill entirely. Two consequences follow: **retrieval precision is a cost control as much as a quality control**, and every tool schema you attach is charged on every single request whether the tool is used or not. Twenty tools attached to every call is a permanent tax.

Why part numbers tokenise badly, and why it matters beyond cost. Tokenisers are trained on general text, so common English words map to single tokens while an alphanumeric string like HX-4471-B is split into fragments that carry no meaning individually. The cost is minor. The retrieval consequence is not: dense embeddings of fragmented identifiers are unreliable, which is a large part of why sparse keyword retrieval remains essential for technical content. Part 2 measures exactly that.

1.2 The context window is not free space

Teams assume a large context window means you can put everything in and let the model sort it out. What actually happens is that accuracy degrades as irrelevant context grows, latency rises roughly with input length, and cost rises directly.

Figure 2 · Lost in the middle — where a passage sits changes whether it is used

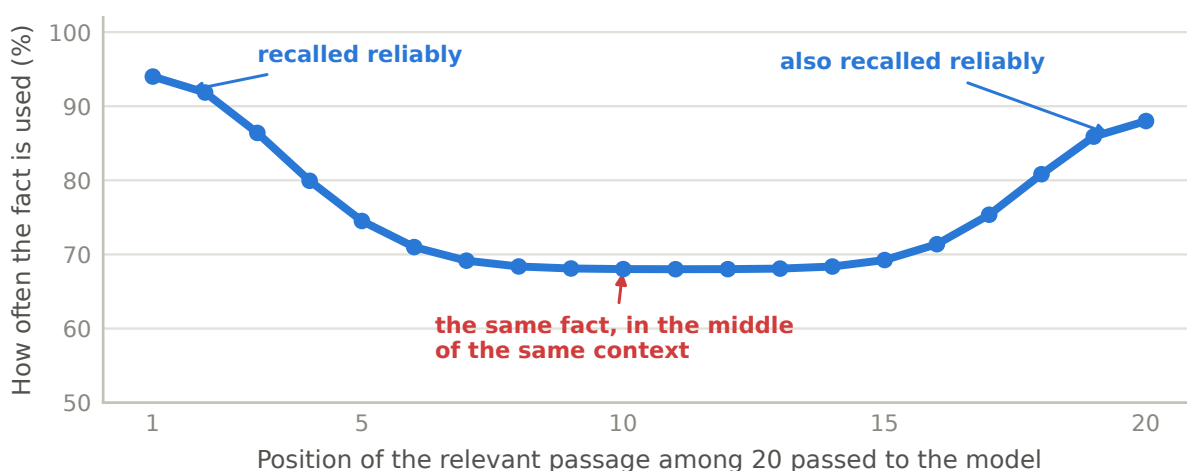


Figure 2. The lost-in-the-middle effect. Across many models, information at the beginning or end of a long context is recalled substantially more reliably than the same information in the middle. Practical consequences: put the most relevant passage first, order by relevance rather than document order, and keep the passage count small enough that ordering matters less.

Figure 3 · Passing twenty passages instead of five costs three times as much and frequently answers worse

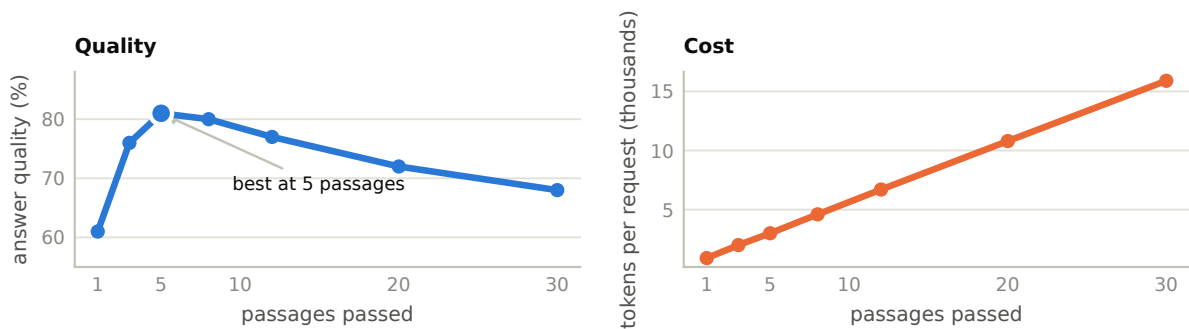


Figure 3. The two effects combined. Quality peaks at a handful of passages and then declines, while cost climbs linearly. This is the single most surprising measurement in Task 1, and teams need to see it on their own data to believe it.

The briefing pack. Give an engineer a two-page brief containing exactly what they need and they will act on it correctly. Give them a two-hundred-page pack containing the same two pages somewhere in the middle and they will miss things — not because they are careless, but because the signal is diluted. The model behaves the same way, and for related reasons.

1.3 Chunking — the decision with the largest quality impact

Strategy	How it works	Good for	Weakness
Fixed size	N tokens with overlap	Uniform prose	Cuts tables and procedures mid-structure
Sentence-aware	Breaks on sentence boundaries	General documents	Ignores document structure
Paragraph	Breaks on paragraph boundaries	Well-structured text	Highly variable chunk sizes
Section-aware	Breaks on headings and hierarchy	Technical manuals	Requires reliable structure extraction
Layout-aware	Respects tables, figures, columns	Industrial documentation	Needs a capable parser
Parent-child	Small chunks to search, large to return	Precision plus completeness	More complex index and retrieval logic
Semantic	Breaks where meaning shifts	Mixed content	Slow to compute, harder to reason about

For service manuals and maintenance procedures, section-aware or parent-child almost always beats fixed size — and it is typically a one-day change.

1.4 Overlap, and the procedure that got cut in half

Overlap exists so that a passage split across a chunk boundary is still retrievable. Ten to fifteen per cent is the usual default. What it mitigates is best understood through the specific failure it prevents.

Figure 4 · The strongest argument for structure-aware chunking

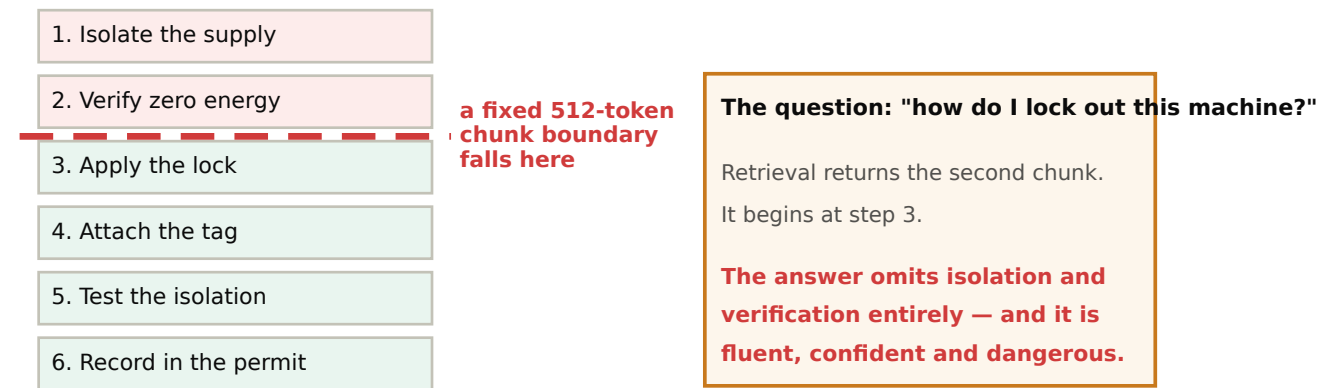


Figure 4. A fixed chunk boundary falls between steps 2 and 3 of a lock-out procedure. A query about locking out the machine retrieves the second chunk, which begins at step 3 — so the answer omits isolation and verification entirely. It is fluent, confident, complete-sounding, and describes an unsafe procedure. Section-aware chunking prevents this; overlap mitigates it.

1.5 Why industrial documents break naive extraction

Feature	What naive extraction does to it
Tables	Returns column text as a run-on sentence. The row-to-value relationship is destroyed, so torque values become dissociated from part numbers.
Multi-column layout	Extracts in the wrong reading order, interleaving two unrelated columns into one incoherent passage.
Figures and diagrams	Discarded entirely, along with the callout numbers that make the surrounding text meaningful.
Headers and footers	Repeated on every page. Left in, they pollute every chunk and dilute embeddings with boilerplate.
Numbered procedures	Flattened. Step numbering and hierarchy are lost, and steps become independent statements.
Scanned pages	Contribute nothing without OCR — and their absence is silent , with no error raised.

Figure 5 · Four levels of extraction, and what each one throws away

Level 1 · Naive text	keeps: words, roughly in order loses: tables · columns · structure · images
Level 2 · Layout-aware	keeps: reading order, paragraph structure loses: table semantics
Level 3 · Table-preserving	keeps: tables as structured rows — row-to-value survives loses: section context
Level 4 · Metadata-rich	keeps: + hierarchy, product family, version, access level loses: nothing that matters today

Moving from level 1 to level 3 typically produces a larger accuracy improvement than any change to the model, the prompt or the embedding.

Figure 5. Each level recovers something the previous one destroyed. Level 4 is not optional if access-filtered retrieval is required, because that needs access level as indexed metadata on every chunk — and adding it later means re-indexing the whole corpus.

1.6 The metadata that earns its place

Field	Enables	Consequence of omitting it
Source document and page	Citation and verification	Answers cannot be checked, so they will not be trusted
Section hierarchy	Parent–child retrieval	Chunks arrive without the heading that gives them meaning
Product family or model	Filtered retrieval	Procedures from the wrong variant returned confidently
Document version and status	Superseded-content suppression	The system cites a withdrawn procedure
Effective and expiry dates	Time-correct answers	Obsolete guidance presented as current
Access level	Role-filtered retrieval	The security model cannot be implemented at all
Language	Correct-language retrieval	Mixed-language results confuse retrieval and generation
Content type	Tuning retrieval by type	Tables and prose treated and ranked identically

Versioning and supersession are a safety concern, not housekeeping. Version every document as a distinct object; mark superseded content explicitly rather than deleting it; filter to current by default with an override for historical queries; make status visible in the citation; trigger re-indexing on publication rather than nightly; and retain superseded content for audit and for equipment still running the older revision.

A retrieval system that confidently cites a withdrawn safety procedure is worse than one that finds nothing. If you take one thing from this section, take document status as an indexed, filterable field.

1.7 Token budgeting

Component	Typical share	How to control it
System prompt and instructions	300 – 1,000	Written once, reviewed, versioned. Rarely the problem
Tool and MCP schemas	0 – 4,000	Attach only tools relevant to this request
Retrieved passages	2,000 – 10,000	The main lever. Rerank, then cut to three to five
Conversation history	0 – 5,000	Summarise older turns rather than carrying them verbatim
User question	20 – 200	Negligible
Reserved for the answer	500 – 2,000	Must be reserved explicitly, or generation truncates

The pattern to repeat all day. "Retrieve twenty, rerank, pass four." Cheaper, faster and measurably more accurate — three improvements from one change. It is the most transferable thing on Day 3, and it reappears in Parts 2 and 4.

2.1 Dense, sparse and hybrid

Method	How it works	Strong on	Weak on
Dense (vector)	Semantic similarity of embeddings	Paraphrase, synonyms, concepts	Exact identifiers — may return a similar-looking part number
Sparse (BM25)	Term matching weighted by rarity	Part numbers, error codes, acronyms	Paraphrase — a differently-phrased question finds nothing
Hybrid	Both, with rank fusion	Both of the above	Two indexes and a fusion step to maintain

Figure 6 · Averaged together the two look similar. Split by question class they are opposites.

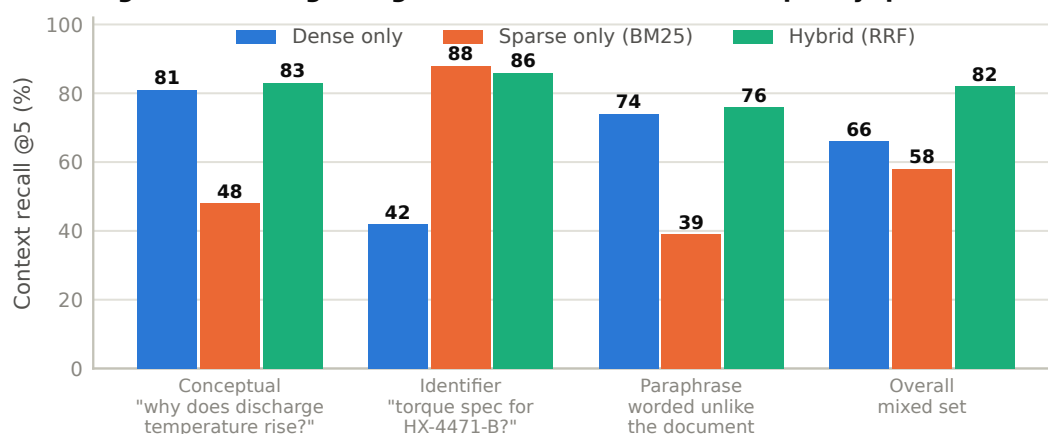


Figure 6. Averaged over a mixed question set the two methods look broadly similar and you learn nothing. Split by question class they are near-opposites — which is why Task 2 insists on reporting every metric per class as well as overall.

The argument that settles it in one sentence. Your engineers type: "what is the torque spec for the mounting bolts on HX-4471-B". Dense retrieval handles the first half and can quietly fail on the second. Sparse handles the second and fails on the first. Nothing about that failure is visible — you get a plausible passage about a different part.

2.2 Reciprocal Rank Fusion

Figure 7 · Reciprocal Rank Fusion — combining two rankings without comparing two scores

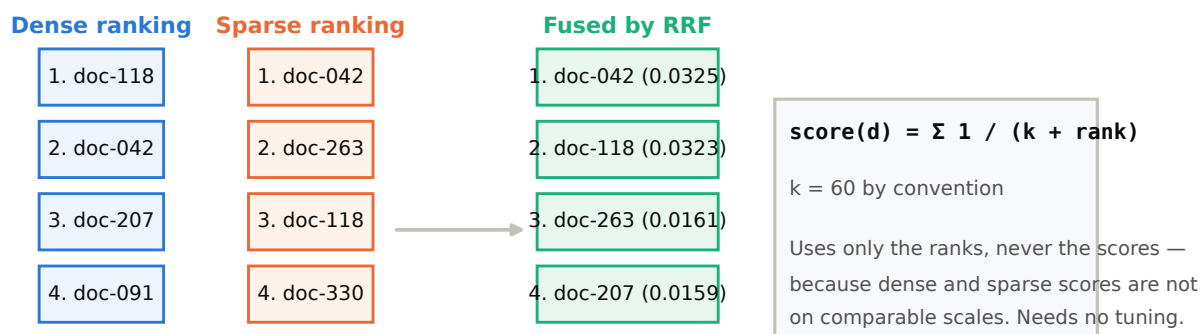


Figure 7. RRF combines two ranked lists using the reciprocal of each item's rank rather than its raw score. It exists because dense and sparse scores are not on comparable scales and normalising them reliably is difficult. It needs no tuning, which is a large part of why it is the default.

2.3 Reranking — the highest-return addition

Retrieve 30 to 50 candidates by hybrid search, rerank them with a cross-encoder, cut to the top three to five, then generate.

Figure 8 · Retrieve broadly, rerank precisely, pass few — three improvements from one change

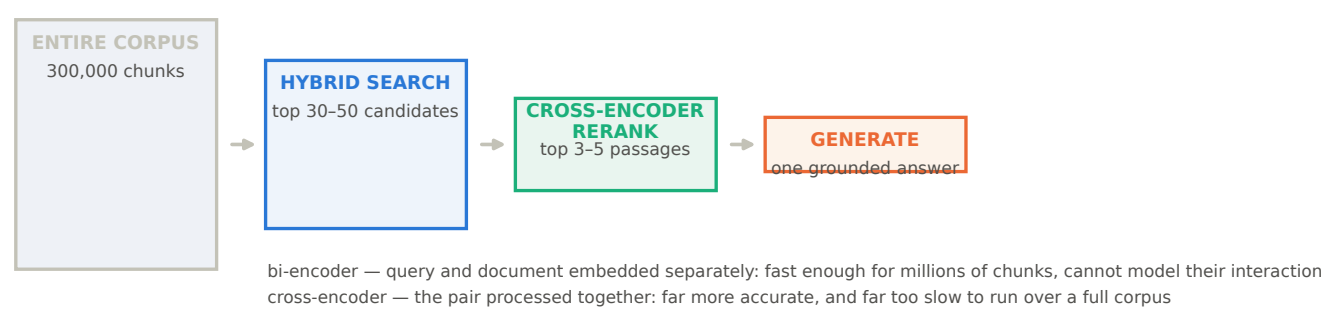


Figure 8. A bi-encoder embeds query and document separately, so it can never model their interaction — and that limitation is exactly why it is fast enough to run over millions of documents. A cross-encoder processes the pair together and models the interaction directly, which is far more accurate and far too slow for a full corpus. Using each where it is strong gives you the recall of the first and the precision of the second.

How to say it. "Reranking is usually the highest return per day of engineering work available to a retrieval system. If you add one thing after this course, add this." It also *reduces* cost, because you pass four passages instead of twenty.

2.4 Filtering, rewriting and expansion

- **Metadata filtering.** Restrict candidates *before* search — product family, version, access level, date. Turns a hard ranking problem into an easy one by removing what should never have been a candidate. Frequently the largest single improvement, and almost always underused, because it needs metadata captured at ingestion.
- **Query rewriting.** Resolve pronouns and conversational references, expand abbreviations, split compound questions. **Measure it** — it does not always help and can distort intent.
- **Query expansion.** Add synonyms and domain terms. Valuable where field vocabulary differs from documentation vocabulary. A domain glossary drives it.

2.5 Structural retrieval patterns

Pattern	How it works	Use when
Parent-child chunking	Search small chunks, return the larger parent	The best general default — precision in search, completeness in the answer
Hierarchical retrieval	Retrieve the section, then within it	Large structured documents
Multi-hop retrieval	One retrieval informs the next query	Questions needing facts from two documents joined
Graph RAG	Traverse entity relationships	Questions about how components or faults relate — powerful, frequently premature
Self-checking RAG	Verify the answer against passages before responding	High-stakes answers where unsupported claims are unacceptable
Retrieval fallback	A second strategy when the first scores weakly	Robustness
No-answer detection	Explicitly return that the corpus does not cover this	Always

Watch for — enthusiasm for knowledge graphs. Graph RAG is genuinely powerful for relationship questions and it is expensive to build and maintain. Hybrid retrieval plus reranking plus metadata filtering usually gets further, faster, on industrial documentation. The question to ask is: what question can you not answer without it?

2.6 No-answer detection

Without it, retrieval always returns its top matches however weak, the model answers anyway, and the user gets a fluent, confident, wrong answer — **with a citation to a document that does not support it**, which makes it worse rather than better.

Figure 9 · No-answer detection — declining is a feature, and it is measurable

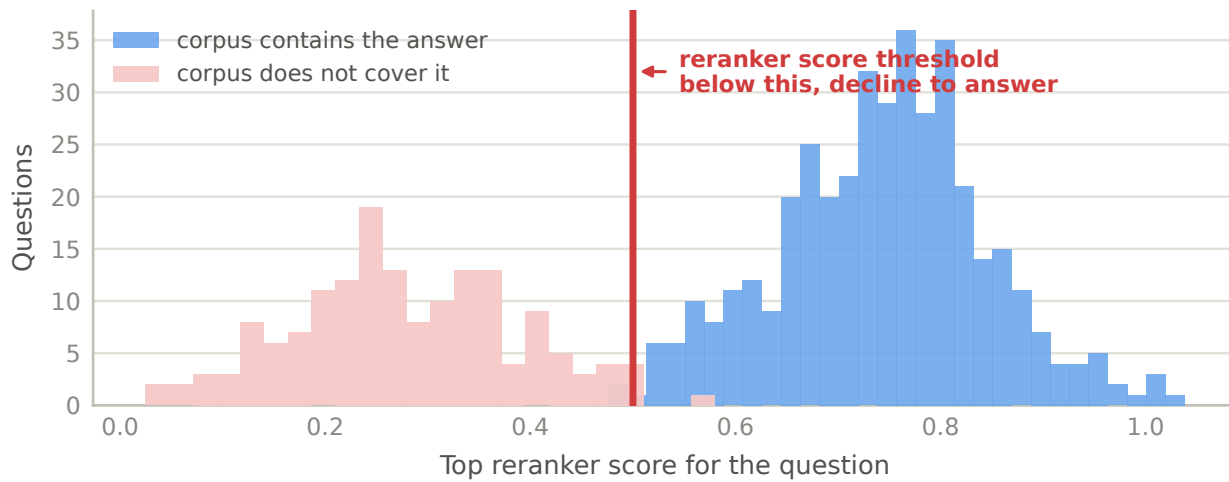


Figure 9. Implementation is two things together: a threshold on the reranker score, and an explicit instruction that the model must answer only from the provided passages and must say so when they are insufficient. Measurement is a set of questions your corpus genuinely does not cover, and the fraction on which the system correctly declines.

The exercise that makes this land. Write ten questions your documentation definitely does not answer — about equipment you do not make, or a procedure that does not exist. Run them. Most teams discover their system answers all ten confidently.

3.1 What MCP is, and the arithmetic that justifies it

Every AI application integrates with the same enterprise systems, and each one writes its own bespoke integration. MCP is a standard protocol between an AI application (the **client**) and a capability provider (the **server**).

Figure 10 · The arithmetic that makes MCP an enterprise decision rather than a technical preference

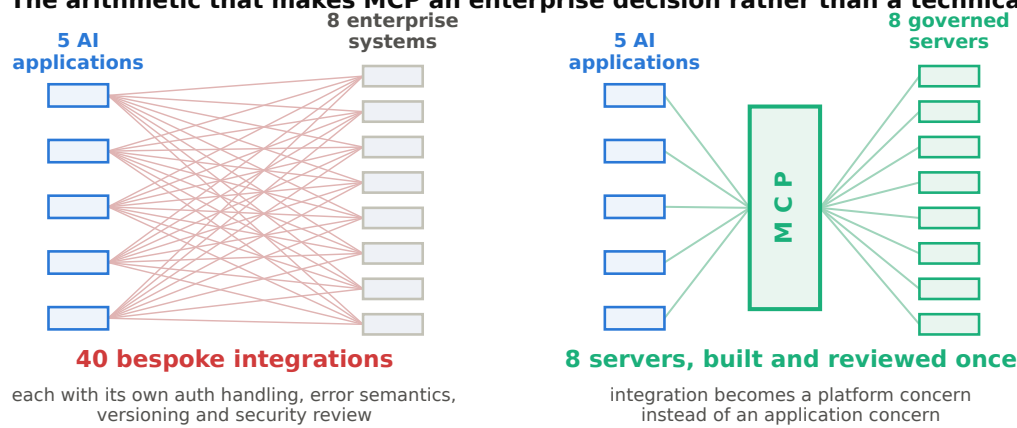


Figure 10. Five applications across eight systems is forty integrations, each with its own authentication handling, error semantics, versioning and security review. Or eight servers, each built and reviewed once, consumed by any client. The team that owns the service-ticket system builds and governs one server, and every AI application consumes it under the same contract.

	MCP client	MCP server
Is	The AI application	A capability provider wrapping a system or data source
Owned by	The application team	The team that owns the underlying system
Responsible for	Deciding when to call, handling results, carrying user identity	Exposing capabilities, validating inputs, enforcing its own access rules

A server can expose three primitives: **tools** (actions the model may invoke), **resources** (data the client may read) and **prompts** (reusable templates). Support varies by host application — several implement tools only, so design your servers so the tool interface alone is sufficient and treat the other two as enhancements.

3.2 When to use MCP, and when not to

Use MCP when	Use a direct call when
The model decides at runtime whether the capability is needed	The step always happens in a fixed pipeline
Several AI applications need the same capability	One application needs it, always the same way
The capability is owned by another team	You own both sides and the contract is stable
You want consistent tool-call logging and governance	The call is already inside a governed service

The distinction that stops overreach. Does the model decide whether this happens? If not, it is not a tool. A retrieval step that always runs before generation is a pipeline stage — wrapping it in MCP adds latency and indirection for nothing.

3.3 Tool contracts — the least-discussed cause of agent failure

The model chooses tools by reading their descriptions. A badly described tool is not called, or is called wrongly, and **neither failure is obvious** — both look like model reasoning problems, so teams change the model and the problem persists.

Element	Requirement	Common failure
Name	Unambiguous and specific	Two tools whose names could each plausibly apply
Description	What it does, when to use it, when not to	Describes the implementation instead of the purpose
Input schema	Typed, each parameter described and constrained	Free-text parameters filled inconsistently
Output schema	Predictable structure	Raw system responses with inconsistent shape
Error semantics	Defined responses for not-found, unauthorised, unavailable	Exceptions, leaving the model with nothing usable
Examples	One or two representative invocations	None — usage inferred from the name alone

The standard to write to. Write tool descriptions for a competent new colleague who has never seen the system. That is precisely the reader you have — and unlike a colleague, it will not ask a clarifying question.

3.4 The six security controls, and the confused deputy

Risk	How it manifests	Mitigation
Over-broad tool scope	A read-everything tool invoked for a restricted user	Least privilege; server-side authorisation per caller
Confused deputy	The server acts with its own high privilege on behalf of a low-privilege user	Propagate caller identity; never a shared service account
Injected tool invocation	Retrieved content induces a tool call	Tool authorisation derives from caller identity, never from content
Untrusted server	A server added without review receives sensitive arguments	Registry of approved servers; review before connection
Argument injection	Model-generated arguments used unsanitised downstream	The server validates and parameterises its own inputs
Silent data egress	A tool sends internal content externally	Explicit egress review per tool; outbound logging

Figure 11 · The classic vulnerability of this architecture, and it is very easily introduced

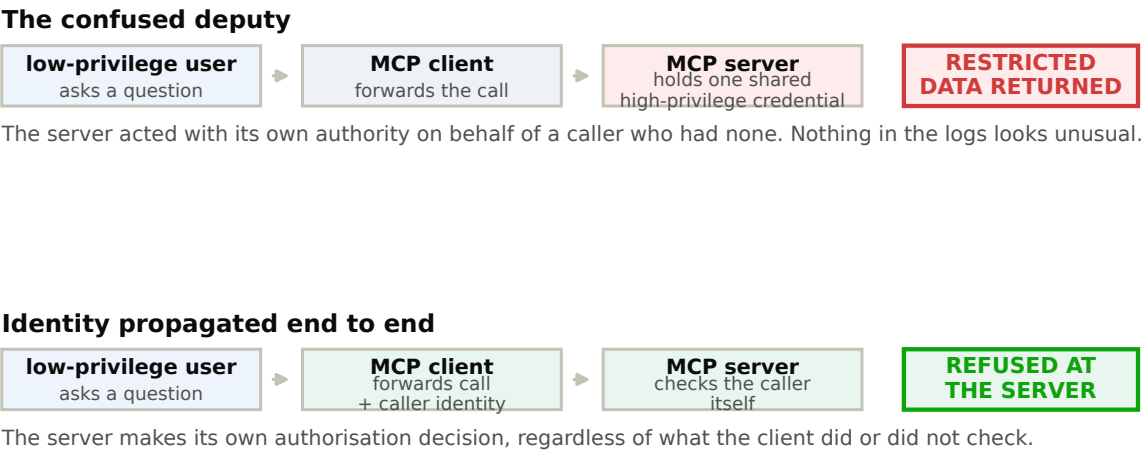


Figure 11. The confused deputy is a privileged component acting on behalf of an unprivileged caller without checking what that caller is allowed to do. It is very easily introduced in MCP architectures, because the natural implementation is for the server to hold one high-privilege credential to the underlying system. Identity must flow end to end, and the server must make its own decision.

4.1 The eight metrics, and what each one diagnoses

The point of having eight rather than one is that they attribute failure to a *stage*. Four are computable from a gold set with no model involved; four need a model as judge. Knowing which is which is what makes the numbers trustworthy.

Metric	Question it answers	Diagnoses
Context recall	Did retrieval find the passages needed?	Retrieval — the usual culprit
Context precision	Of what was retrieved, how much was relevant?	Retrieval — cost and lost-in-the-middle
Faithfulness	Is the answer supported by the passages?	Generation — hallucination
Answer relevance	Does the answer address the question?	Generation — drift and evasion
Citation accuracy	Do citations actually support the claims?	Generation — the trust signal users check
Hallucination rate	How often are unsupported claims made?	Generation — what management asks about
No-answer accuracy	Does it decline correctly when coverage is absent?	System — almost never measured
Groundedness	Is every claim traceable to a source?	System — composite quality signal

4.2 The diagnosis table — two numbers, one answer

Figure 12 · Two numbers, one diagnosis

	Context recall LOW	Context recall HIGH
Faithfulness HIGH	RETRIEVAL FAILED the answer never had a chance fix: chunking, hybrid, reranking, filtering	THE SYSTEM IS WORKING measure cost and latency; examine failing slices
Faithfulness LOW	RETRIEVAL FAILED (same diagnosis — faithfulness cannot rescue it) fix: retrieval	FOUND AND IGNORED the right passages were retrieved fix: prompt grounding, citation enforcement

The overwhelming majority of real cases land in the left column — retrieval failed and the model was blamed. Teams routinely spend weeks on prompt engineering for a problem a reranker would have fixed in a day.

Figure 12. This converts "the RAG system gives bad answers" into a specific diagnosis in one step. A fourth case sits outside the grid: high recall, high faithfulness, and the answer is still wrong — which means the *source document* is wrong or outdated. That is a content problem solved by versioning and supersession, not a system problem.

4.3 Building the evaluation set — six rules

- Real questions first**, from support tickets, engineer queries and search logs. Invented questions are systematically easier than real ones.
- Record the correct passage, not just the answer.** This is what makes context recall measurable, and what separates a diagnostic evaluation set from a score you cannot act on.
- Include out-of-corpus questions** — ten to twenty the documentation genuinely does not cover.
- Include per-role variants.** If retrieval is access-filtered, the correct answer differs by role.
- Include known-hard cases** — ambiguity, multi-document questions, superseded content.
- Synthetic questions to fill gaps**, with an expert verifying every correct answer.

4.4 Traces and spans

A **trace** is the full record of one request; a **span** is one step within it. Eight spans are worth recording: query received · query rewrite · retrieval · reranking · context assembly · tool calls · generation · post-processing.

The span to insist on. Context assembly records exactly what was passed to the model, with token counts. When a user reports a wrong answer, that span usually answers the question immediately — the passage was not there at all, or it was there in position fourteen of twenty. Almost no proof of concept records it. It is cheap to add and it will resolve most of your support questions in one look.

4.5 The observability dashboard, and one signal worth more than it looks

Eight panels, each with an alert condition: quality · no-answer behaviour · retrieval health · tool calls · cost · latency · feedback · error clusters. Two of the alert conditions are counter-intuitive and worth stating explicitly.

Alert when the decline rate falls toward zero. A system that has stopped saying "I don't know" has not got better. Something in retrieval or thresholding has changed, and it is now answering questions it should refuse.

Tokens per request is a quality signal, not just a cost signal. A sudden rise usually means retrieval quality dropped — scores fell, so the system pulled in more passages to compensate, or a fallback path started firing. It is one of the earliest warnings available and it needs **no labelled data at all**, which makes it operationally valuable in a way most quality metrics are not.

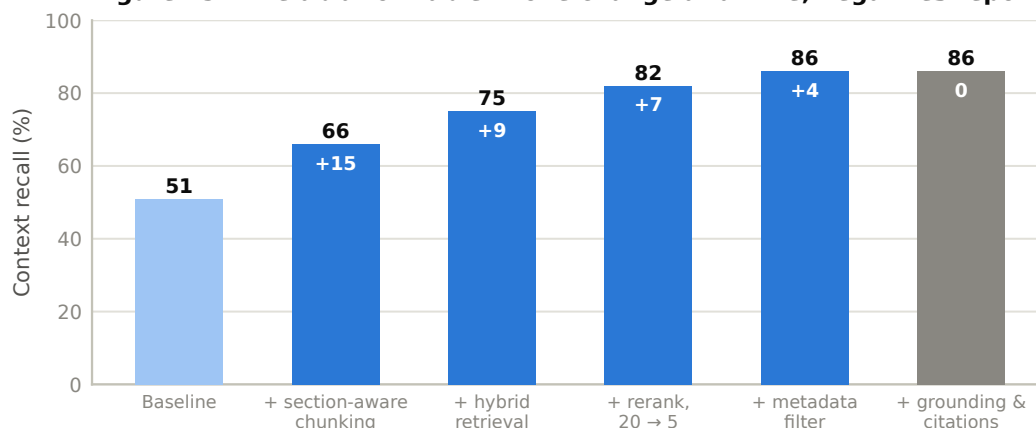
4.6 Regression testing, and why the suite only grows

Triggered by any prompt, model, embedding, chunking or retrieval change, plus a scheduled weekly run regardless. The suite contains the gold set, every fixed production failure, out-of-corpus questions, per-role variants and known-hard cases.

The compounding argument. Every production failure that gets fixed becomes a permanent case. Within a year the suite encodes everything the system has ever got wrong, and no regression can silently reintroduce a past failure. That is the difference between a system that improves and one that oscillates.

4.7 The ablation, and why negatives matter

Figure 13 · The ablation table — one change at a time, negatives reported



Reranking also cut cost from 10.8k to 3.1k tokens per query — the only step that improved quality, latency and cost together. Grounding and citations gave no measurable recall gain on this data. Reported anyway — a change measured at zero is a change to remove.

Figure 13. One change at a time, with the delta attributed to each. Change four things at once and performance improves, and you have learned almost nothing — you cannot tell which one to keep and which is adding complexity for free. A change measured at zero is a change to *remove*, and finding one makes the system simpler, cheaper and faster at no measurable cost.

4.8 Prompt grounding and citation enforcement

Prompt grounding instructs the model to answer only from the provided passages, to say so when they are insufficient, and not to use general knowledge. Cheap and effective — and it **shifts the rate rather than eliminating the behaviour**, so treat it as one

control among several.

Citation enforcement requires every claim to carry a reference, and then **validates programmatically** that the cited passage was retrieved and does contain the claim. The validation step is what makes it real; unvalidated citations are frequently plausible and wrong, which is worse than none.

The adoption argument — use this with management. An engineer who can check the source will use a system that is right 85% of the time. An engineer who cannot check will not use one that is right 95% of the time, because a wrong answer they cannot detect is worse than no answer. **Citation accuracy is an adoption metric as much as a quality metric.**

Retrieval metrics — computable from a gold set, no model needed

context recall = fraction of questions whose gold passage appears in the top k context precision
= fraction of retrieved passages that are relevant no-answer accuracy = fraction of out-of-corpus
questions correctly declined

Compute context recall first. It diagnoses the stage where most failures actually happen.

Reciprocal Rank Fusion

$\text{score}(d) = \sum \text{over lists } 1 / (k + \text{rank of } d \text{ in that list})$ k = 60 by convention

Uses ranks, never scores — dense and sparse scores are not on comparable scales. Needs no tuning.

The retrieval pipeline that works

hybrid search → 30–50 candidates cross-encoder rerank → top 3–5 metadata filter applied BEFORE
search, not after generate with grounding + validated citations decline if the top reranker score
is below threshold

Token budget — the six components

system prompt 300 – 1,000 written once, versioned tool schemas 0 – 4,000 charged on EVERY request
retrieved passages 2,000 – 10,000 the main lever history 0 – 5,000 summarise older turns question
20 – 200 negligible reserved for answer 500 – 2,000 must be reserved explicitly

The diagnosis, in two numbers

recall LOW → retrieval failed; fix chunking, hybrid, rerank, filter recall HIGH, faithful LOW →
passages found and ignored; fix grounding and citations recall HIGH, faithful HIGH, answer still
wrong → the source document is wrong; fix versioning recall HIGH, faithful HIGH, answer correct →
working; measure cost, latency and failing slices

The eight spans to record per request

query received · query rewrite · retrieval · reranking context assembly · tool calls · generation
· post-processing

Context assembly is the one almost nobody records and the one that answers most support questions.

Tool contract — six elements, all required

name · description (including when NOT to use) · input schema output schema · error semantics ·
one or two examples

Term	Plain meaning
Bi-encoder	Embeds query and document separately. Fast enough for millions of chunks; cannot model their interaction.
BM25	Sparse retrieval by term matching weighted by rarity. Strong on part numbers and error codes.
Chunk	The unit of text that gets embedded and retrieved. How you cut them is the largest single quality decision in ingestion.
Citation enforcement	Every claim carries a reference, and the reference is validated programmatically against what was retrieved.
Context assembly	Building what the model actually sees. The span worth recording above all others.
Context precision	Of what was retrieved, how much was relevant. Drives cost and lost-in-the-middle.
Context recall	Did retrieval find the passage that answers the question? The number that diagnoses most failures.
Cross-encoder	Processes query and document together. Far more accurate, far too slow for a full corpus — hence two stages.
Dense retrieval	Semantic similarity between embeddings. Strong on paraphrase, weak on exact identifiers.
Faithfulness	Is the answer supported by the passages it was given? Distinguishes generation failures from retrieval ones.
Groundedness	Is every claim traceable to a source? A composite quality signal.
Hybrid retrieval	Dense and sparse together, combined by rank fusion. Close to mandatory for technical content.
Lost in the middle	Information at the start or end of a long context is recalled far more reliably than the same information in the middle.
MCP	A standard protocol between an AI application and a capability provider. Turns integration from an application concern into a platform concern.
No-answer detection	Explicitly returning that the corpus does not cover the question. A feature, and measurable.
Overlap	Repeating text across chunk boundaries so a split passage is still retrievable. 10–15% is usual.
Parent–child chunking	Search small chunks, return the enclosing section. Precision in search, completeness in the answer.
Prompt grounding	Instructing the model to answer only from the provided passages. Shifts the rate; does not eliminate the behaviour.
Rank fusion (RRF)	Combining ranked lists by reciprocal rank rather than by score. Needs no tuning.
Reranking	A second, more accurate scoring pass over a small candidate set. The best return per day of work available.
Span	One step within a request. Eight are worth recording.
Supersession	Marking a document as withdrawn and replaced, and filtering it out of retrieval by default. A safety concern.
Token	Roughly three-quarters of a word, and much less in technical text. The unit you are billed in.
Trace	The full record of one request, made of spans.
Confused deputy	A privileged component acting for an unprivileged caller without checking what that caller may do.

If you remember five things. Most retrieval failures are retrieval failures. Ingestion quality caps everything downstream. Retrieve broadly, rerank precisely, pass few. Measure retrieval and generation separately. And declining to answer is a feature — a system that never says "I don't know" has not been measured on questions it cannot answer.